Peer Skills Lab – Überblick für die Übergabe

Diese Anleitung erklärt in einfachen Worten, was unsere Webseite kann, wo sie liegt und was man tun muss, damit sie weiterläuft. Technische Details stehen nur dort, wo eine Entwickler:in sie braucht.

Stand: Oktober 2026

---

1. Was ist die Webseite?

Eine Buchungsplattform für unsere Kurse. Es gibt drei Arten von Nutzer:innen:

Rolle	Das kann sie
Studierende	Kurse ansehen, buchen und absagen, nach dem Kurs Feedback geben, eigene Statistik sehen
Tutor:innen	Kurse anlegen (auch aus Vorlagen), Teilnehmerlisten führen, Anwesenheit eintragen, Räume reservieren, Feedback zu den eigenen Kursen lesen
Admins	Alle Nutzer:innen und Kurse verwalten, Gesamtstatistiken ansehen

Ohne Login sind diese Seiten öffentlich: «Über uns», «Tutor:in werden», FAQ, Impressum und Datenschutz.

---

2. Wo liegt was?

Was	Wo	Wozu
Die Webseite (live)	Microsoft Azure, Region Schweden	Hier läuft sie für alle Nutzer:innen
Die Datenbank	Microsoft Azure	Hier stehen alle Nutzer:innen, Kurse und Buchungen
Der Programmcode	GitHub: `peerskillslab/Booking`	Hier liegt der gesamte Code samt Verlauf aller Änderungen

Webadresse der Seite: https://peerskills.agreeableground-97fa1669.swedencentral.azurecontainerapps.io

Das Azure-Konto läuft über **info@peerskillslab.ch**.

---

3. Was muss sicher aufbewahrt werden?

Diese Zugänge brauchen alle, die die Seite weiterbetreuen. Bitte nicht per Mail oder Chat herumschicken, sondern in einem Passwort-Manager ablegen:

- Login für **Azure** (info@peerskillslab.ch)
- Login für **GitHub** (Zugang zum Code)
- **Datenbank-Passwort** und **Login-Schlüssel (JWT_SECRET)**: Sie wurden beim allerersten Aufsetzen einmal angezeigt. Wer sie nicht mehr hat, kann sie im Azure-Portal bei der Container App unter «Secrets» nachsehen.

---

4. Häufige Aufgaben

Einen Text auf der Webseite ändern

Das ist Sache einer Person, die mit Code umgehen kann (oder Claude Code). Texte stehen in `peer-skills-lab-kurse/src/pages/`, je Seite eine Datei, zum Beispiel `TutorWerden.jsx` für die Seite «Tutor:in werden» oder `FAQ.jsx` für die FAQ. Nach der Änderung muss die Seite neu veröffentlicht werden (siehe nächster Punkt).

Änderungen live schalten

Auf einem Computer mit Docker Desktop und angemeldeter Azure-Verbindung:

```
bash deploy-azure-local.sh update
```

Das dauert einige Minuten. Eine automatische Veröffentlichung gibt es nicht, jede Änderung muss so von Hand live geschaltet werden.

Eine neue Admin-Person anlegen

Zuerst muss die Person ein normales Konto auf der Webseite erstellen. Danach kann jemand mit Admin-Rechten ihre Rolle im Admin-Bereich ändern. Das allererste Admin-Konto wird einmalig per Befehl angelegt:

```
az containerapp exec --name peerskills --resource-group peerskills-rg \
  --command "node create-admin.js admin@beispiel.ch 'Name' passwort"
```

Die Seite lokal ausprobieren

Nur für Entwickler:innen: `./dev-start.sh` startet eine Testversion auf dem eigenen Computer, `./dev-stop.sh` beendet sie. Dafür braucht es eine Datenbank und eine Datei `server/.env` mit den Zugangsdaten.

---

5. Was man wissen sollte

- **Keine automatischen Tests.** Nach jeder Änderung die Seite kurz im Browser durchklicken.
- **Es läuft immer nur eine Instanz.** Bei sehr vielen gleichzeitigen Nutzer:innen könnte die Seite langsam werden.
- **Passwort-Reset-Mails** funktionieren nur, wenn ein Mail-Server (SMTP) eingetragen ist. Das ist aktuell nicht automatisch eingerichtet.
- **Registrierung für alle offen.** Jede gültige E-Mail-Adresse kann ein Konto erstellen, es gibt keine Prüfung auf die Uni-Adresse.
- **Herkunft des Codes.** Die Seite wurde ursprünglich mit der Plattform «Base44» gebaut und später auf ein eigenes System umgezogen. Darum heissen manche Dinge im Code noch anders, als man erwarten würde.

---

6. Für Entwickler:innen: kurze Technik-Übersicht

- **Frontend:** React mit Vite, im Ordner `peer-skills-lab-kurse/`
- **Backend:** Node.js mit Express, im Ordner `server/`, Datenbank PostgreSQL
- **Ein Docker-Container** enthält beides und läuft in Azure Container Apps
- **Datenbankstruktur:** `server/db/schema.sql`, wird bei jedem Serverstart automatisch angewendet. Änderungen nur additiv (`IF NOT EXISTS`).
- **Rechte:** Wer was darf, steht in `server/middleware/authorize.js`. Der Schutz der Daten liegt im Backend, nicht im Frontend.
- **Neue Funktion einbauen:** zuerst Backend (`server/routes/`), dann Seite in `src/pages/`, dann Route in `src/App.jsx` eintragen.
- **Tutor:in verknüpfen:** immer über `instructor_email`, nie über den Namen.